The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch21_llm_training

AYuSh

Contents

Chapter 21: LLM Training & Alignment

A modern LLM is built in stages: pretraining teaches everything and aligns nothing; the alignment stack — SFT, then preference optimization — turns a text predictor into an assistant; and a bag of efficiency techniques (PEFT, quantization, MoE) determines what any of it costs. This chapter runs the *entire* pipeline in miniature on Chapter 20's pretrained checkpoints, so every claim in the alignment story arrives with a number attached — including the failure modes (reward hacking, forgetting, expert collapse) that interviews increasingly probe, because production teams hit them weekly.

The measurements: scaling laws fit from a trained grid — excess loss falls as a power law in parameters (slope −0.13) and data (−0.37), the Chinchilla logic in miniature (Listing 1); SFT with the prompt-masking decision measured *both ways* — masking wins task accuracy (0.812 vs 0.734) but collapses general perplexity 12× because the unmasked LM loss was acting as replay (Listing 2); a reward model at 95% pairwise accuracy whose maximum-reward input, found by greedy search, is *gibberish scoring above every real document* (Listing 3); policy-gradient RLHF where the KL coefficient is a dial — reward +5.11/+4.51/+3.26 against KL 17/7/1.6 as β rises, fluency visibly returning (Listing 4); DPO moving preference margin from 0 to +6.4 and pushing dispreferred-text perplexity 74 → 174 with one supervised-looking loss — including the sign bug that silently trained the *opposite* preference first (Listing 5); LoRA matching full fine-tuning (0.766 vs 0.764) with **1.0%** of trainable parameters and 2.7× less forgetting (Listing 6); quantization measured down the bit ladder — INT8 free, INT4 per-channel 145 vs per-tensor 171 ppl, INT3 divergence, and a 20× outlier channel multiplying per-tensor error 8.5× (Listing 7); and a mixture-of-experts layer collapsing to one expert without a load-balancing loss, recovering accuracy with it (Listing 8).

Pretraining and scaling laws

Pretraining is Chapter 20's CLM objective at industrial scale, and the science of it is *allocation*: given a compute budget $C \approx 6ND$ (parameters × tokens × 6 FLOPs), how big a model on how much data? Kaplan et al. (2020) found loss falls as a power law in N, D, and C — smooth, predictable, extrapolable — and recommended big models on modest data. **Chinchilla** (Hoffmann et al., 2022) redid the measurement with better-tuned baselines and reversed the advice: compute-optimal training wants N and D scaled *together*, roughly **20 tokens per parameter** — GPT-3-era models were 4× oversized for their data. Listing 1 reproduces the shape of the law at miniature scale: a trained grid gives excess loss $\propto N^{-0.13}$ at fixed data and $\propto D^{-0.37}$ at fixed parameters (real-scale exponents are ~0.34/0.28; the miniature's ordering — data-starved regime, data helps more — is itself the Chinchilla point), with the 300-sequence run *overfitting past the uniform baseline*, a reminder that the laws describe the underfit regime. The post-

Chinchilla footnote interviews now expect: production models (Llama-class) train far *beyond* 20 tok/param — compute-optimal ≠ inference-optimal, because training cost is paid once and serving cost forever, so overtraining a smaller model is the economically correct move. Data reality to mention: dedup, quality filtering, and mixture weights (code/web/books) move loss as much as architecture; epochs > ~4 on the same data yield rapidly diminishing returns.

Supervised fine-tuning

SFT teaches *format and behavior*: instruction-response pairs, loss on the response. Listing 2 stages the standard implementation decision — mask the prompt or not — and gets a two-sided answer worth more than the usual one-liner. Masking the loss to the answer token concentrates gradient on the behavior: best task accuracy (0.812 vs 0.734 unmasked vs 0.326 base). But with *nothing else* in the loss, general LM perplexity explodes 133 → 1,645 — while the unmasked run, "wasting" gradient on re-predicting prompt tokens, held at 169: the prompt loss was functioning as **replay regularization**. Production practice does both deliberately: mask prompts *and* mix pretraining data into the SFT batches (1-10%), exactly because this trade is real. Everything else about instruction tuning is data curation — quality beats quantity (LIMA's thousand examples), diversity of instructions drives generalization to unseen tasks (FLAN's finding), and Chapter 20's loss-masking bug (training on prompts) remains the most common SFT implementation error in the wild.

Reward models and reward hacking

RLHF's first component converts human judgments into a differentiable signal: collect pairwise preferences (rankings are easier and more reliable than absolute scores), then train a **Bradley-Terry** model — $P(A \succ B) = \sigma(r(A) - r(B))$, loss $-\log \sigma(r_{chosen} - r_{rejected})$ — with the reward head on a pretrained encoder. Listing 3 does it: 95% pairwise accuracy on fresh pairs after 300 steps. Then the listing does what interviews want you to know happens next: it *optimizes against* the reward model — greedy coordinate ascent over input tokens — and finds a gibberish string scoring **+7.69 against +7.32 for the best real document**. The RM is a proxy trained on-distribution; outside that distribution its extrapolations are unconstrained, and any optimizer pointed at it will exit the distribution immediately — Goodhart's law with gradients. (The miniature adds a comic detail: the adversarial string is full of *hockey* vocabulary — the RM's decision boundary is so warped off-manifold that anti-topic tokens maximize a pro-space reward.) This is why reward models are refreshed with on-policy data between RL rounds, why ensembles and uncertainty penalties exist, and why the KL leash of the next listing is not optional.

RLHF: optimization with a leash

The RLHF objective is $\max_{\pi} \mathbb{E}[r(x)] - \beta\,\mathrm{KL}(\pi \| \pi_{ref})$ — maximize reward *without leaving the reference model's distribution*. Listing 4 implements the honest miniature (REINFORCE with a moving baseline rather than full PPO — same gradient family, minus clipping) and sweeps β: at β=0 the policy gains +4.2 reward but drifts to KL 17 and emits team-name salad — it found Listing 3's off-distribution exploit, *as theory predicts*; β=0.15 gives +3.6 at KL 7; β=0.6 gives +2.4 at KL 1.6 with visibly fluent samples. The dial *is* the concept: β trades reward against distributional fidelity, and every production system tunes it (plus early stopping on KL). What PPO adds at scale: clipped importance ratios for sample reuse, a learned value function as baseline, per-token KL — machinery for variance and stability, not a different objective. **RLAIF** swaps the human preference source for an AI judge (constitutional AI: critiques and revisions guided by principles) — same pipeline, cheaper labels, inheriting the judge's biases instead of annotator noise.

DPO

DPO (Rafailov et al., 2023) collapses the RM + RL pipeline into one loss by observing that the KL-constrained optimum has a closed form ($\pi^* \propto \pi_{ref} e^{r/\beta}$), so the reward can be *reparameterized* in terms of the policy itself: $r(x) = \beta \log \frac{\pi(x)}{\pi_{ref}(x)}$. Substituting into Bradley-Terry gives a supervised loss on preference pairs — $-\log \sigma(\beta[(\log \pi/\pi_{ref})_{chosen} - (\log \pi/\pi_{ref})_{rejected}])$ — no reward model, no sampling loop, no baseline. Listing 5 implements it (the gradient weight $(1 - \sigma(z))$ — largest on wrongly-ranked pairs — falls out on one line) and measures: implicit-reward margin 0 → +6.38, held-out pair accuracy 0.865, dispreferred-text perplexity pushed 74 → 174 while preferred pays a mild tax 70 → 87. That last asymmetry is the known DPO artifact stated honestly: the loss optimizes the *margin*, and chosen-sequence likelihood can fall too (motivating IPO, cDPO, and reference-free variants). The listing also preserves a debugging war story: a sign error in the gradient trained the *opposite* preference — margin −35.8 — and the tell was dispreferred perplexity *improving*; preference losses fail silently in the direction you least expect, so always plot the margin. Trade-offs vs RLHF for interviews: DPO is offline (no exploration — it only reweights behaviors present in the pairs), simpler and more stable; PPO-style RLHF can discover new behaviors and exploit per-token credit, at 4× the infrastructure (policy, reference, RM, value).

Parameter-efficient fine-tuning

Full fine-tuning of an N-billion-parameter model costs ~16N bytes of optimizer state (fp32 weights + Adam moments); PEFT attacks that. **LoRA**: freeze W_0, learn $W = W_0 + \frac{\alpha}{r} AB$ with $A \in \mathbb{R}^{d \times r}$, $B \in \mathbb{R}^{r \times d}$, $r \ll d$ — task updates are empirically low-rank, so a rank-4..64 correction captures them. Listing 6 measures the whole claim on attention Q/V matrices: accuracy 0.766 vs full fine-tuning's 0.764 with **1,536 of 152,064**

parameters trained (1.0%), and — the underrated half — the frozen backbone *forgets less*: post-tuning perplexity 217 vs 584. Deployment detail that closes the pitch: merge AB into W_0 once and inference is exactly a dense model — zero added latency, or keep adapters separate and hot-swap many tasks on one backbone. **QLoRA** extends it: backbone quantized to NF4 (a 4-bit format shaped to the normal distribution of weights), LoRA in bf16 on top, gradients flowing through dequantization — 65B fine-tuning on one GPU. The rest of the family, one line each: adapters (small bottleneck MLPs inserted per layer — add latency), prefix/prompt tuning (learn virtual tokens; cheapest, weakest, and where "soft prompts" live), $(IA)^3$ (learned per-channel scalings). Practical LoRA knobs: which matrices (Q,V classic; all-linear better at low rank), r and α (quality saturates fast in r), and LR ~10× higher than full FT.

Quantization

Serving cost is bytes moved, so weights get smaller. Listing 7 walks the bit ladder with round-to-nearest absmax on the real checkpoint: **INT8 is free** (ppl 133 vs 132.9 — eight bits comfortably exceed weight SNR), **INT4 is a real trade** (144.8 per-channel vs 171.2 per-tensor — granularity is the whole game at low bits), **INT3 diverges** (205/375). The mechanism behind per-channel's win, isolated: inject one 20× outlier channel and per-tensor error multiplies 8.5× — one hot channel sets the scale for everyone, crushing everyone else's resolution — while per-channel isolates it. That single experiment explains the production landscape: **LLM.int8** keeps outlier channels in fp16 (activation outliers, same physics); **GPTQ** quantizes column-by-column, spreading each column's error over not-yet-quantized weights via the Hessian; **AWQ** chooses scales from *activation* statistics (protect the weights that multiply large activations); **GGUF** is llama.cpp's packaging of grouped k-quants for CPU serving; NF4 (QLoRA) shapes the codebook to the weight distribution. The taxonomy to keep straight: weight-only (memory-bound decoding — the common case) vs weight+activation (compute-bound prefill, INT8 tensor cores); post-training quantization (all the above) vs quantization-aware training (train through the rounding, rarely needed above 4 bits).

Mixture of experts

MoE decouples parameters from FLOPs: replace the FFN with E experts and a router that sends each token to the top-k (usually 1-2) — total capacity scales with E, per-token compute doesn't. Listing 8 builds top-1 routing and demonstrates the failure that defines the technique: **without a load-balancing loss, routing collapses** — usage [0, 1, 0, 0], one expert does everything (rich-get-richer: the trained expert wins more tokens, trains more, wins more), and accuracy sits at what one small expert can do (0.952). Adding the Switch-Transformer auxiliary loss ($E\sum_e f_e \bar{p}_e$ — minimized by uniform routing) revives capacity: accuracy 0.970, though two experts stay dead — honest miniature of a real difficulty (production MoEs add router noise, careful init, capacity factors with token

dropping, and still monitor expert utilization). The accounting the listing prints — 32% of parameters active per token — is the entire business case: Mixtral 8×7B holds 47B parameters and spends ~13B per token; Switch, GLaM, DeepSeek-MoE scale the same trick. Costs to volunteer: all experts must live in memory (VRAM scales with total, not active), all-to-all communication when experts shard across devices, and fine-tuning instability relative to dense models.

Code implementations

(These listings reuse `common.py` — the batched miniature transformer of Chapter 20's Listing 1, gradient-checked to 5×10^{-10} *— and Chapter 20's pretrained CLM/MLM checkpoints as base models.)*

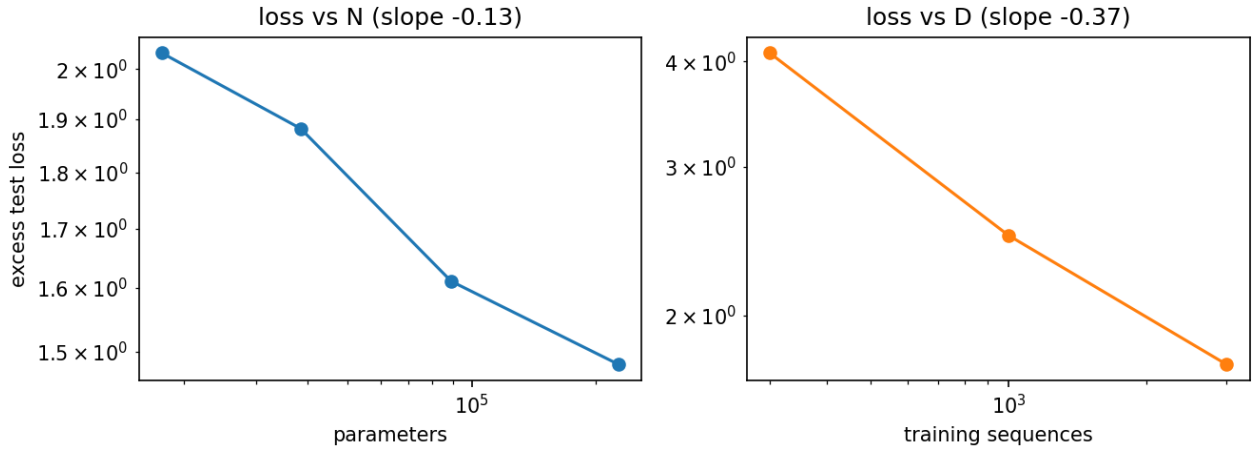
Listing 1 — Scaling laws in miniature: loss vs parameters and data

```
"""Listing 1: scaling laws in miniature -- loss vs parameters and data follows power
laws.
   (resumable: reruns continue where the grid left off)"""
import os, time, json, numpy as np
from common import Tiny, load_corpus, softmax
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt

X, y, vocab = load_corpus(vmax=1000, seqlen=16); V = len(vocab)
perm = np.random.default_rng(0).permutation(len(X)); X = X[perm]
Xte = X[-800:]

def train_eval(d, n_data, steps=700, seed=0):
    rng = np.random.default_rng(seed)
    m = Tiny(V, 16, d=d, H=max(2, d//16), depth=2, causal=True, seed=seed)
    Xtr = X[:n_data]
    for _ in range(steps):
        xb = Xtr[rng.integers(0, len(Xtr), 32)]
        hf, c = m.fwd(xb)
        Pr = softmax(hf@m.P['Wu'])
        tgt = np.roll(xb, -1, 1); sel = (tgt > 3); sel[:, -1] = False
        dlog = Pr.copy(); dlog[sel, tgt[sel]] -= 1; dlog[~sel] = 0; dlog /= sel.sum()
        g = m.bwd(dlog@m.P['Wu'].T, c, {'Wu': hf.reshape(-1,m.d).T@dlog.reshape(-1,V)})
        m.adam(g, 1.5e-3)
    lls, cnt = 0.0, 0
    for s in range(0, len(Xte), 200):
        xb = Xte[s:s+200]; hf, _ = m.fwd(xb)
        Pr = softmax(hf@m.P['Wu'])
        tgt = np.roll(xb, -1, 1); sel = (tgt > 3); sel[:, -1] = False
        lls += np.log(Pr[sel, tgt[sel]]+1e-12).sum(); cnt += sel.sum()
    nparams = sum(v.size for v in m.P.values())
    return nparams, -lls/cnt

DONE = 'scal_done.json'
res = json.load(open(DONE)) if os.path.exists(DONE) else {}
jobs = [("N", d, 5000) for d in [8, 16, 32, 64]] + [("D", 32, nd) for nd in [300,
1000, 3000]]
t0 = time.time()
for kind, d, nd in jobs:
    key = f"{kind}_{d}_{nd}"
    if key in res: continue
    if time.time() - t0 > 25: print("...rerun to continue"); break
    npar, L = train_eval(d, nd)
    res[key] = [npar, L]; json.dump(res, open(DONE, 'w'))
```

```
    print(f"{kind}-scaling  d={d:3d}  data={nd:5d}: params {npar:8,d}  test loss {L:.
3f}")
if len(res) == len(jobs):
    print("\nGRID COMPLETE")
    Ns = [(res[f"N_{d}_5000"][0], res[f"N_{d}_5000"][1]) for d in [8, 16, 32, 64]]
    a_n = np.polyfit(np.log([n for n,_ in Ns]), np.log([l-3.7 for _,l in Ns]), 1)[0]
    Ds = [(nd, res[f"D_32_{nd}"][1]) for nd in [300, 1000, 3000]]
    a_d = np.polyfit(np.log([n for n,_ in Ds]), np.log([l-3.7 for _,l in Ds]), 1)[0]
    print(f"power-law exponent, loss vs PARAMS (fixed data): {a_n:.2f}")
    print(f"power-law exponent, loss vs DATA (fixed params): {a_d:.2f}")
    fig, axes = plt.subplots(1, 2, figsize=(9, 3.4))
    axes[0].loglog([n for n,_ in Ns], [l-3.7 for _,l in Ns], 'o-');
axes[0].set(xlabel='parameters', ylabel='excess test loss', title=f'loss vs N (slope
{a_n:.2f})')
    axes[1].loglog([n for n,_ in Ds], [l-3.7 for _,l in Ds], 'o-', c='tab:orange');
axes[1].set(xlabel='training sequences', title=f'loss vs D (slope {a_d:.2f})')
    fig.tight_layout(); fig.savefig("/sessions/intelligent-modest-faraday/mnt/ADVANCE
RAG/AI_ML_Interview_Book/manuscript/figures/ch21/fig_l1_scaling.png", dpi=150)
    print("figure saved: fig_l1_scaling.png")
```

Output:

```
N-scaling  d=  8  data= 5000: params    17,664  test loss 5.733
N-scaling  d= 16  data= 5000: params    38,400  test loss 5.582
N-scaling  d= 32  data= 5000: params    89,088  test loss 5.311
N-scaling  d= 64  data= 5000: params   227,328  test loss 5.180
D-scaling  d= 32  data=  300: params    89,088  test loss 7.796
D-scaling  d= 32  data= 1000: params    89,088  test loss 6.189
D-scaling  d= 32  data= 3000: params    89,088  test loss 5.450
GRID COMPLETE
power-law exponent, loss vs PARAMS (fixed data): -0.13
power-law exponent, loss vs DATA (fixed params): -0.37
figure saved: fig_l1_scaling.png
```



Listing 2 — Supervised fine-tuning: instruction format and prompt masking

```
"""Listing 2: supervised fine-tuning -- instruction format, and what loss-masking the
prompt changes."""
import numpy as np
from common import Tiny, load_corpus, softmax
rng = np.random.default_rng(2)

X, y, vocab = load_corpus(vmax=1000, seqlen=16); V = len(vocab)
```

```
inv = {i: w for w, i in vocab.items()}
perm = np.random.default_rng(0).permutation(len(X)); X, y = X[perm], y[perm]
ANS = [vocab["space"], vocab["hockey"]]                          # answer tokens exist in
the vocab
Q = vocab.get("topic", 1)

def format_ex(seq, label):
    """instruction format: [13 content tokens][topic][ANSWER][pad] -- answer at
position 14"""
    return np.concatenate([seq[:13], [Q, ANS[label], 0]])
Xf = np.array([format_ex(s, l) for s, l in zip(X, y)])
tr, te = Xf[:2000], Xf[-1500:]; yte = y[-1500:]

def sft(mask_prompt, iters=350):
    m = Tiny(V, 16, causal=True, seed=0)
    m.P = {k: v.copy() for k, v in np.load('clm_ckpt.npz', allow_pickle=True)
['P'].item().items()}
    m.opt = {k: [np.zeros_like(v), np.zeros_like(v)] for k, v in m.P.items()}
    r = np.random.default_rng(0)
    for it in range(iters):
        xb = tr[r.integers(0, len(tr), 32)]
        hf, c = m.fwd(xb)
        Pr = softmax(hf@m.P['Wu'])
        tgt = np.roll(xb, -1, 1)
        if mask_prompt:
            sel = np.zeros_like(xb, bool); sel[:, 13] = True   # loss ONLY on the
answer token
        else:
            sel = (tgt > 3); sel[:, -1] = False                # loss on everything
(the bug)
        dlog = Pr.copy(); dlog[sel, tgt[sel]] -= 1; dlog[~sel] = 0; dlog /= sel.sum()
        g = m.bwd(dlog@m.P['Wu'].T, c, {'Wu': hf.reshape(-1,m.d).T@dlog.reshape(-1,V)})
        m.adam(g, 5e-4)
    return m

def answer_acc(m):
    ok = []
    for s in range(0, len(te), 250):
        xb = te[s:s+250]
        hf, _ = m.fwd(xb)
        logits = hf[:, 13]@m.P['Wu']                           # prediction AT the
'topic' position
        ok.append(np.array(ANS)[np.argmax(logits[:, ANS], 1)] == xb[:, 14])
    return np.concatenate(ok).mean()

base = Tiny(V, 16, causal=True, seed=0)
base.P = np.load('clm_ckpt.npz', allow_pickle=True)['P'].item()
print(f"base CLM (no SFT), answer accuracy:            {answer_acc(base):.3f}")
m_all = sft(False); print(f"SFT, loss on ALL tokens (prompt unmasked):
{answer_acc(m_all):.3f}")
m_msk = sft(True);  print(f"SFT, loss on ANSWER only (prompt masked):
{answer_acc(m_msk):.3f}")

# side effect: what did each do to general LM quality? perplexity on plain text
def ppl(m):
    lls, cnt = 0.0, 0
    for s in range(0, 800, 200):
        xb = X[-800:][s:s+200]; hf, _ = m.fwd(xb)
        Pr = softmax(hf@m.P['Wu'])
        tgt = np.roll(xb, -1, 1); sel = (tgt > 3); sel[:, -1] = False
        lls += np.log(Pr[sel, tgt[sel]]+1e-12).sum(); cnt += sel.sum()
    return np.exp(-lls/cnt)
print(f"plain-text perplexity: base {ppl(base):.0f}   unmasked-SFT {ppl(m_all):.0f}
masked-SFT {ppl(m_msk):.0f}")
print("two-sided lesson: masking concentrates gradient on the target behavior (best
answer accuracy)")
print("but with NOTHING else in the loss, general LM ability collapses 12x -- unmasked
```

```
loss acted as replay.")
print("production SFT masks prompts AND mixes in pretraining data to keep both.")
```

Output:

```
base CLM (no SFT), answer accuracy:            0.326
SFT, loss on ALL tokens (prompt unmasked):     0.734
SFT, loss on ANSWER only (prompt masked):      0.812
plain-text perplexity: base 133   unmasked-SFT 169   masked-SFT 1645
two-sided lesson: masking concentrates gradient on the target behavior (best answer
accuracy)
but with NOTHING else in the loss, general LM ability collapses 12x -- unmasked loss
acted as replay.
production SFT masks prompts AND mixes in pretraining data to keep both.
```

Listing 3 — A reward model: Bradley-Terry training, and reward hacking on camera

```
"""Listing 3: a reward model -- Bradley-Terry training, and reward hacking on
camera."""
import numpy as np
from common import Tiny, load_corpus, softmax
rng = np.random.default_rng(3)

X, y, vocab = load_corpus(vmax=1000, seqlen=16); V = len(vocab)
perm = np.random.default_rng(0).permutation(len(X)); X, y = X[perm], y[perm]

# preference data: pairs of chunks; raters "prefer" the on-topic (space) one.
def make_pairs(n):
    sp = X[y == 0]; hk = X[y == 1]
    i = rng.integers(0, len(sp), n); j = rng.integers(0, len(hk), n)
    return sp[i], hk[j]
Ap, Bp = make_pairs(1200)                                    # A preferred over B

# reward model: the MLM encoder + scalar head, trained on Bradley-Terry: -log sig(r(A)
- r(B))
rm = Tiny(V, 16, causal=False, seed=0)
rm.P = {k: v.copy() for k, v in np.load('mlm_ckpt.npz', allow_pickle=True)
['P'].item().items()}
rm.opt = {k: [np.zeros_like(v), np.zeros_like(v)] for k, v in rm.P.items()}
w = np.random.default_rng(0).normal(0, .05, rm.d)
sig = lambda z: 1/(1+np.exp(-np.clip(z, -30, 30)))
def score(m, wv, Xs):
    out = []
    for s in range(0, len(Xs), 250):
        hf, _ = m.fwd(Xs[s:s+250]); out.append(hf.mean(1)@wv)
    return np.concatenate(out)
r = np.random.default_rng(0)
for it in range(300):
    i = r.integers(0, 1000, 16)
    xa, xb = Ap[i], Bp[i]
    ha, ca = rm.fwd(xa); hb, cb = rm.fwd(xb)
    ra = ha.mean(1)@w; rbv = hb.mean(1)@w
    p = sig(ra - rbv)
    d = (p - 1)/len(i)                                       # d loss / d (ra - rb)
    gw = (ha.mean(1)*d[:, None]).sum(0) - (hb.mean(1)*d[:, None]).sum(0)
    ga = rm.bwd(np.repeat((d[:, None]*w[None])[:, None, :], 16, 1)/16, ca)
    gb = rm.bwd(np.repeat((-d[:, None]*w[None])[:, None, :], 16, 1)/16, cb)
    rm.adam({k: ga[k]+gb[k] for k in ga}, 3e-4); w -= 0.05*gw
At, Bt = make_pairs(400)
acc = (score(rm, w, At) > score(rm, w, Bt)).mean()
print(f"reward model pairwise accuracy on fresh pairs: {acc:.3f}")
```

```
# reward hacking: search for the highest-reward INPUT. is it good text -- or the
confound?
print("\nmean reward by candidate type:")
for name, c in [("on-topic real text (space)", X[y == 0][:400]),
                ("off-topic real text (hockey)", X[y == 1][:400]),
                ("random token strings", rng.integers(4, V, (400, 16)))]:
    print(f"  {name:30s}: {score(rm, w, c).mean():+.2f}")

# now OPTIMIZE against the RM: greedy coordinate ascent over tokens, starting from
noise
inv = {i2: wd for wd, i2 in vocab.items()}
adv = rng.integers(4, V, (1, 16))
for sweep in range(2):
    for pos in range(16):
        trial = np.repeat(adv, 60, 0); trial[:, pos] = rng.integers(4, V, 60)
        sc = score(rm, w, trial)
        adv = trial[sc.argmax():sc.argmax()+1]
print(f"\nadversarial input after greedy search: reward {score(rm, w, adv)[0]:+.2f}"
      f"  (best real space text ~ {score(rm, w, X[y == 0][:400]).max():+.2f})")
print("text:", " ".join(inv[t] for t in adv[0]))
print("~95% pairwise accuracy ON-distribution, yet gibberish out-scores every real
document --")
print("this is the objection to naive reward maximization, and why RLHF needs a KL
leash (Listing 4)")
```

Output:

```
reward model pairwise accuracy on fresh pairs: 0.948

mean reward by candidate type:
  on-topic real text (space)    : +3.40
  off-topic real text (hockey)  : -3.08
  random token strings          : +3.82

adversarial input after greedy search: reward +7.69  (best real space text ~ +7.32)
text: mon found stevens anonymous young he's njd edmonton pp young watch flames young
canucks pts fuel
~95% pairwise accuracy ON-distribution, yet gibberish out-scores every real document --
this is the objection to naive reward maximization, and why RLHF needs a KL leash
(Listing 4)
```

Listing 4 — RLHF in miniature: policy gradient against the RM, with and without the KL leash

```
"""Listing 4: RLHF in miniature -- policy-gradient against the reward model, with and
without the KL leash."""
import numpy as np
from common import Tiny, load_corpus, softmax
rng = np.random.default_rng(4)

X, y, vocab = load_corpus(vmax=1000, seqlen=16); V = len(vocab)
inv = {i: w for w, i in vocab.items()}

# frozen reward model from Listing 3's recipe (retrained quickly here for self-
containedness)
rm = Tiny(V, 16, causal=False, seed=0)
rm.P = {k: v.copy() for k, v in np.load('mlm_ckpt.npz', allow_pickle=True)
['P'].item().items()}
rm.opt = {k: [np.zeros_like(v), np.zeros_like(v)] for k, v in rm.P.items()}
wrm = np.random.default_rng(0).normal(0, .05, rm.d)
sp, hk = X[y == 0], X[y == 1]
```

```
sig = lambda z: 1/(1+np.exp(-np.clip(z, -30, 30)))
r0 = np.random.default_rng(0)
for it in range(250):
    i = r0.integers(0, len(sp), 16); j = r0.integers(0, len(hk), 16)
    xa, xb = sp[i], hk[j]
    ha, ca = rm.fwd(xa); hb, cb = rm.fwd(xb)
    d = (sig(ha.mean(1)@wrm - hb.mean(1)@wrm) - 1)/16
    gw = (ha.mean(1)*d[:, None]).sum(0) - (hb.mean(1)*d[:, None]).sum(0)
    ga = rm.bwd(np.repeat((d[:, None]*wrm[None])[:, None, :], 16, 1)/16, ca)
    gb = rm.bwd(np.repeat((-d[:, None]*wrm[None])[:, None, :], 16, 1)/16, cb)
    rm.adam({k: ga[k]+gb[k] for k in ga}, 3e-4); wrm -= 0.05*gw
def reward(Xs):
    hf, _ = rm.fwd(Xs); return hf.mean(1)@wrm

base = Tiny(V, 16, causal=True, seed=0)
base.P = np.load('clm_ckpt.npz', allow_pickle=True)['P'].item()
PROMPT = [vocab.get("the", 1)]

def rollout(m, B=16, T=12):
    """sample continuations + keep per-step logprobs of chosen tokens (and base
logprobs for KL)"""
    seqs = np.tile(PROMPT, (B, 1))
    for t in range(T):
        pad = np.zeros((B, 16-seqs.shape[1]), int)
        hf, _ = m.fwd(np.concatenate([seqs, pad], 1))
        logits = hf[:, seqs.shape[1]-1]@m.P['Wu']; logits[:, :4] = -1e9
        p = softmax(logits)
        nxt = np.array([rng.choice(V, p=pi) for pi in p])
        seqs = np.concatenate([seqs, nxt[:, None]], 1)
    return np.concatenate([seqs, np.zeros((B, 16-seqs.shape[1]), int)], 1)

def logprobs(m, seqs, upto):
    hf, c = m.fwd(seqs)
    P = softmax(hf@m.P['Wu'])
    tgt = np.roll(seqs, -1, 1)
    lp = np.log(P[np.arange(len(seqs))[:, None], np.arange(16)[None], tgt] + 1e-12)
    return lp[:, :upto], (hf, c, P, tgt)

def train_policy(beta, iters=60):
    m = Tiny(V, 16, causal=True, seed=0)
    m.P = {k: v.copy() for k, v in np.load('clm_ckpt.npz', allow_pickle=True)
['P'].item().items()}
    m.opt = {k: [np.zeros_like(v), np.zeros_like(v)] for k, v in m.P.items()}
    T = 12; baseline = 0.0
    for it in range(iters):
        seqs = rollout(m)
        R = reward(seqs)
        lp_pi, (hf, c, P, tgt) = logprobs(m, seqs, T)
        lp_ref, _ = logprobs(base, seqs, T)
        kl = (lp_pi - lp_ref).sum(1)                          # sequence-level KL
estimate
        Rt = R - beta*kl                                      # KL-penalized reward
        baseline = .9*baseline + .1*Rt.mean()
        adv = (Rt - baseline)
        # REINFORCE: d/dtheta -adv * sum log pi(a_t) => at logits: adv*(P - onehot) on
generated steps
        dlog = P.copy()
        dlog[np.arange(len(seqs))[:, None], np.arange(16)[None], tgt] -= 1
        msk = np.zeros((len(seqs), 16)); msk[:, :T] = 1
        dlog *= (adv[:, None]*msk/msk.sum())[:, :, None]   # dLoss/dlogits for loss = -
adv*logpi
        g = m.bwd(dlog@m.P['Wu'].T, c, {'Wu': hf.reshape(-1,m.d).T@dlog.reshape(-1,V)})
        m.adam(g, 4e-4)
    seqs = rollout(m, 64)
    lp_pi, _ = logprobs(m, seqs, T); lp_ref, _ = logprobs(base, seqs, T)
    return reward(seqs).mean(), (lp_pi-lp_ref).sum(1).mean(), seqs

r_base = reward(rollout(base, 64)).mean()
```

```
print(f"base policy: mean reward {r_base:+.2f}")
for beta, tag in [(0.0, "no KL penalty "), (0.15, "beta = 0.15   "), (0.6, "beta =
0.6    ")]:
    R, kl, seqs = train_policy(beta)
    print(f"{tag}: reward {R:+.2f}   KL-to-base {kl:6.1f}   sample: {' '.join(inv[t]
for t in seqs[0][:13])}")
```

Output:

```
base policy: mean reward +0.90
no KL penalty : reward +5.11   KL-to-base   17.0   sample: the san jose w norris devils
center pit devils sharks ottawa adams against
beta = 0.15   : reward +4.51   KL-to-base    7.0   sample: the toronto played for the
world senators los la islanders norris king win
beta = 0.6    : reward +3.26   KL-to-base    1.6   sample: the first bay people here in
first period kings said they pay over
```

Listing 5 — DPO: preference optimization with no reward model and no rollouts

```
"""Listing 5: DPO -- preference optimization with no reward model and no rollouts."""
import numpy as np
from common import Tiny, load_corpus, softmax
rng = np.random.default_rng(5)

X, y, vocab = load_corpus(vmax=1000, seqlen=16); V = len(vocab)
perm = np.random.default_rng(0).permutation(len(X)); X, y = X[perm], y[perm]
sp, hk = X[y == 0], X[y == 1]                                  # preferred: space;
dispreferred: hockey
Wsp, Whk = sp[:1200], hk[:1200]; Tsp, Thk = sp[1200:1600], hk[1200:1600]

base = Tiny(V, 16, causal=True, seed=0)
base.P = np.load('clm_ckpt.npz', allow_pickle=True)['P'].item()

def seq_logprob(m, xb, want_cache=False):
    hf, c = m.fwd(xb)
    P = softmax(hf@m.P['Wu'])
    tgt = np.roll(xb, -1, 1); sel = (tgt > 3); sel[:, -1] = False
    lp = (np.log(P[np.arange(len(xb))[:, None], np.arange(16)[None], tgt] +
1e-12)*sel).sum(1)
    return (lp, (hf, c, P, tgt, sel)) if want_cache else lp

pi = Tiny(V, 16, causal=True, seed=0)
pi.P = {k: v.copy() for k, v in base.P.items()}
pi.opt = {k: [np.zeros_like(v), np.zeros_like(v)] for k, v in pi.P.items()}
BETA = 0.5
sigm = lambda z: 1/(1+np.exp(-np.clip(z, -30, 30)))

def margin(m, A, B):
    return ((seq_logprob(m, A) - seq_logprob(base, A)) - (seq_logprob(m, B) -
seq_logprob(base, B)))
print(f"pre-DPO margin (pi == ref, so identically zero): {margin(pi, Tsp, Thk).mean():
+.3f}")

r = np.random.default_rng(0)
for it in range(120):
    i = r.integers(0, len(Wsp), 16)
    xw, xl = Wsp[i], Whk[i]
    lp_w, (hw, cw, Pw, tw, sw) = seq_logprob(pi, xw, True)
    lp_l, (hl, cl, Pl, tl, sl) = seq_logprob(pi, xl, True)
    ref_w = seq_logprob(base, xw); ref_l = seq_logprob(base, xl)
    z = BETA*((lp_w - ref_w) - (lp_l - ref_l))
```

```
    coef = (1 - sigm(z))*BETA/len(i)                          # |dLoss/dz|: large when
the pair is WRONGLY ranked
    def dlogits(P, tgt, sel, cf):
        d = P.copy()
        d[np.arange(len(P))[:, None], np.arange(16)[None], tgt] -= 1
        return d*sel[:, :, None]*cf[:, None, None]
    dw = dlogits(Pw, tw, sw, coef)                            # dL/dlogits_w = coef*(P-
onehot): descent RAISES lp_w
    dl = dlogits(Pl, tl, sl, -coef)                           # and LOWERS lp_l
    gw_ = pi.bwd(dw@pi.P['Wu'].T, cw, {'Wu': hw.reshape(-1,pi.d).T@dw.reshape(-1,V)})
    gl_ = pi.bwd(dl@pi.P['Wu'].T, cl, {'Wu': hl.reshape(-1,pi.d).T@dl.reshape(-1,V)})
    pi.adam({k: gw_[k]+gl_[k] for k in gw_}, 5e-4)
mg = margin(pi, Tsp, Thk)
print(f"post-DPO: margin {mg.mean():+.3f}   pair accuracy {(mg > 0).mean():.3f}")

def ppl(m, Xs):
    lp = np.concatenate([seq_logprob(m, Xs[s:s+200]) for s in range(0, len(Xs), 200)])
    toks = np.concatenate([((np.roll(Xs[s:s+200], -1, 1) > 3)[:, :-1]).sum(1)+0 for s
in range(0, len(Xs), 200)])
    return np.exp(-lp.sum()/max(1, toks.sum()))
print(f"perplexity on PREFERRED-domain text: base {ppl(base, Tsp):.0f} -> DPO {ppl(pi,
Tsp):.0f}")
print(f"perplexity on DISPREFERRED text:     base {ppl(base, Thk):.0f} -> DPO {ppl(pi,
Thk):.0f}")
print("one supervised-looking loss moved probability mass from rejected to chosen -- no
RM, no sampling loop")
```

Output:

```
pre-DPO margin (pi == ref, so identically zero): +0.000
post-DPO: margin +6.380   pair accuracy 0.865
perplexity on PREFERRED-domain text: base 70 -> DPO 87
perplexity on DISPREFERRED text:     base 74 -> DPO 174
one supervised-looking loss moved probability mass from rejected to chosen -- no RM, no
sampling loop
```

Listing 6 — LoRA: rank-4 adapters vs full fine-tuning

```
"""Listing 6: LoRA -- rank-4 adapters vs full fine-tuning: quality, parameter count,
forgetting."""
import numpy as np
from common import Tiny, load_corpus, softmax
rng = np.random.default_rng(6)

X, y, vocab = load_corpus(vmax=1000, seqlen=16); V = len(vocab)
perm = np.random.default_rng(0).permutation(len(X)); X, y = X[perm], y[perm]
Xte, yte = X[-1500:], y[-1500:]

def load_base():
    m = Tiny(V, 16, causal=True, seed=0)
    m.P = {k: v.copy() for k, v in np.load('clm_ckpt.npz', allow_pickle=True)
['P'].item().items()}
    m.opt = {k: [np.zeros_like(v), np.zeros_like(v)] for k, v in m.P.items()}
    return m
def ppl(m):
    lls, cnt = 0.0, 0
    for s in range(0, 800, 200):
        xb = X[-800:][s:s+200]; hf, _ = m.fwd(xb)
        Pr = softmax(hf@m.P['Wu'])
        tgt = np.roll(xb, -1, 1); sel = (tgt > 3); sel[:, -1] = False
        lls += np.log(Pr[sel, tgt[sel]]+1e-12).sum(); cnt += sel.sum()
    return np.exp(-lls/cnt)
def acc(m, Wc):
```

```
    ok = []
    for s in range(0, len(Xte), 250):
        hf, _ = m.fwd(Xte[s:s+250]); ok.append((hf.mean(1)@Wc).argmax(1) ==
yte[s:s+250])
    return np.concatenate(ok).mean()

N_LAB, RANK, ITERS = 300, 4, 350
targets = [f'{w}{i}' for i in range(2) for w in ('Wq', 'Wv')]  # adapt attention Q and
V, both blocks

def train(mode):
    m = load_base()
    W0 = {k: m.P[k].copy() for k in targets}
    lora = {k: [np.random.default_rng(1).normal(0, .02, (m.d, RANK)), np.zeros((RANK,
m.d))] for k in targets}
    opt = {k: [[np.zeros_like(a), np.zeros_like(a)], [np.zeros_like(b),
np.zeros_like(b)]] for k, (a, b) in lora.items()}
    Wc = np.random.default_rng(1).normal(0, .05, (m.d, 2)); r =
np.random.default_rng(1); t = 0
    for it in range(ITERS):
        if mode == 'lora':
            for k in targets: m.P[k] = W0[k] + lora[k][0]@lora[k][1]
        i = r.integers(0, N_LAB, 24); xb, yb = X[i], y[i]
        hf, c = m.fwd(xb); pool = hf.mean(1)
        Pr = softmax(pool@Wc)
        d = Pr.copy(); d[np.arange(24), yb] -= 1; d /= 24
        g = m.bwd(np.repeat((d@Wc.T)[:, None, :], 16, 1)/16, c)
        Wc -= 0.05*pool.T@d
        t += 1
        if mode == 'full':
            m.adam(g, 5e-4)
        else:                                                  # LoRA: only A, B
receive updates
            for k in targets:
                A, Bm = lora[k]
                gA, gB = g[k]@Bm.T, A.T@g[k]
# chain rule through W = W0 + A B
                for arr, gr, st in [(A, gA, opt[k][0]), (Bm, gB, opt[k][1])]:
                    mo, vo = st
                    mo[:] = .9*mo + .1*np.clip(gr, -1, 1); vo[:] = .999*vo + .
001*np.clip(gr, -1, 1)**2
                    arr -= 2e-3*(mo/(1-.9**t))/(np.sqrt(vo/(1-.999**t))+1e-8)
    if mode == 'lora':
        for k in targets: m.P[k] = W0[k] + lora[k][0]@lora[k][1]
        n_train = sum(a.size + b.size for a, b in lora.values())
    else:
        n_train = sum(v.size for v in m.P.values())
    return m, Wc, n_train

base = load_base(); total = sum(v.size for v in base.P.values())
print(f"base model: {total:,} params   plain-text perplexity {ppl(base):.0f}")
for mode in ['full', 'lora']:
    m, Wc, n = train(mode)
    print(f"{mode:5s} fine-tune ({N_LAB} labels): acc {acc(m, Wc):.3f}   trained {n:,}
params "
          f"({100*n/total:.1f}%)   ppl after {ppl(m):.0f}")
print("rank-4 adapters on Wq/Wv match full fine-tuning at ~1% of the trainable
parameters --")
print("and the frozen backbone forgets less. merge W0 + A·B at deploy time: zero
inference overhead.")
```

Output:

```
base model: 152,064 params   plain-text perplexity 133
full  fine-tune (300 labels): acc 0.764   trained 152,064 params (100.0%)   ppl after
584
```

```
lora  fine-tune (300 labels): acc 0.766   trained 1,536 params (1.0%)   ppl after 217
rank-4 adapters on Wq/Wv match full fine-tuning at ~1% of the trainable parameters --
and the frozen backbone forgets less. merge W0 + A·B at deploy time: zero inference
overhead.
```

Listing 7 — Weight quantization: INT8/INT4, per-tensor vs per-channel, outliers

```python
"""Listing 7: weight quantization -- INT8/INT4, per-tensor vs per-channel, and the
outlier problem."""
import numpy as np
from common import Tiny, load_corpus, softmax
rng = np.random.default_rng(7)

X, y, vocab = load_corpus(vmax=1000, seqlen=16); V = len(vocab)
X = X[np.random.default_rng(0).permutation(len(X))]

def quantize(W, bits, per_channel):
    """symmetric absmax round-to-nearest"""
    q = 2**(bits-1) - 1
    s = (np.abs(W).max(axis=0, keepdims=True) if per_channel else np.abs(W).max())/q
    return np.round(W/(s+1e-12)).clip(-q, q)*s

def ppl(P):
    m = Tiny(V, 16, causal=True, seed=0); m.P = P
    lls, cnt = 0.0, 0
    for s in range(0, 800, 200):
        xb = X[-800:][s:s+200]; hf, _ = m.fwd(xb)
        Pr = softmax(hf@m.P['Wu'])
        tgt = np.roll(xb, -1, 1); sel = (tgt > 3); sel[:, -1] = False
        lls += np.log(Pr[sel, tgt[sel]]+1e-12).sum(); cnt += sel.sum()
    return np.exp(-lls/cnt)

base = np.load('clm_ckpt.npz', allow_pickle=True)['P'].item()
mats = [k for k, v in base.items() if v.ndim == 2 and k not in ('pos',)]
print(f"quantizing {len(mats)} weight matrices; activations and positions kept fp
(weight-only quantization)")
print(f"fp32 baseline perplexity: {ppl(base):.1f}")
for bits in [8, 4, 3]:
    for pc in [False, True]:
        P = {k: (quantize(v, bits, pc) if k in mats else v) for k, v in base.items()}
        print(f"INT{bits}, {'per-channel' if pc else 'per-tensor '}: ppl {ppl(P):
8.1f}")

# the outlier problem (LLM.int8's motivation): one hot channel stretches the scale for
everyone
W = base['W10'].copy()
print(f"\nchannel absmax spread in block-0 FFN W1: {np.abs(W).max(0).min():.2f} ..
{np.abs(W).max(0).max():.2f}")
W[:, 7] *= 20                                          # inject one outlier
channel
e_pt = np.abs(quantize(W, 4, False) - W).mean()
e_pc = np.abs(quantize(W, 4, True) - W).mean()
print(f"after injecting a 20x outlier channel, INT4 mean abs error: per-tensor {e_pt:.
4f}   per-channel {e_pc:.4f}")
print("per-tensor: the outlier sets ONE scale and crushes everyone else's resolution;
per-channel isolates it.")
print("GPTQ adds error compensation (quantize a column, spread its error over the rest
via the Hessian);")
print("AWQ picks scales from ACTIVATION statistics; llm.int8 keeps outlier channels in
fp16")
```

Output:

```
quantizing 14 weight matrices; activations and positions kept fp (weight-only
quantization)
fp32 baseline perplexity: 132.9
INT8, per-tensor : ppl    133.0
INT8, per-channel: ppl    132.8
INT4, per-tensor : ppl    171.2
INT4, per-channel: ppl    144.8
INT3, per-tensor : ppl    374.9
INT3, per-channel: ppl    205.1

channel absmax spread in block-0 FFN W1: 0.29 .. 0.68
after injecting a 20x outlier channel, INT4 mean abs error: per-tensor 0.1352   per-
channel 0.0160
per-tensor: the outlier sets ONE scale and crushes everyone else's resolution; per-
channel isolates it.
GPTQ adds error compensation (quantize a column, spread its error over the rest via the
Hessian);
AWQ picks scales from ACTIVATION statistics; llm.int8 keeps outlier channels in fp16
```

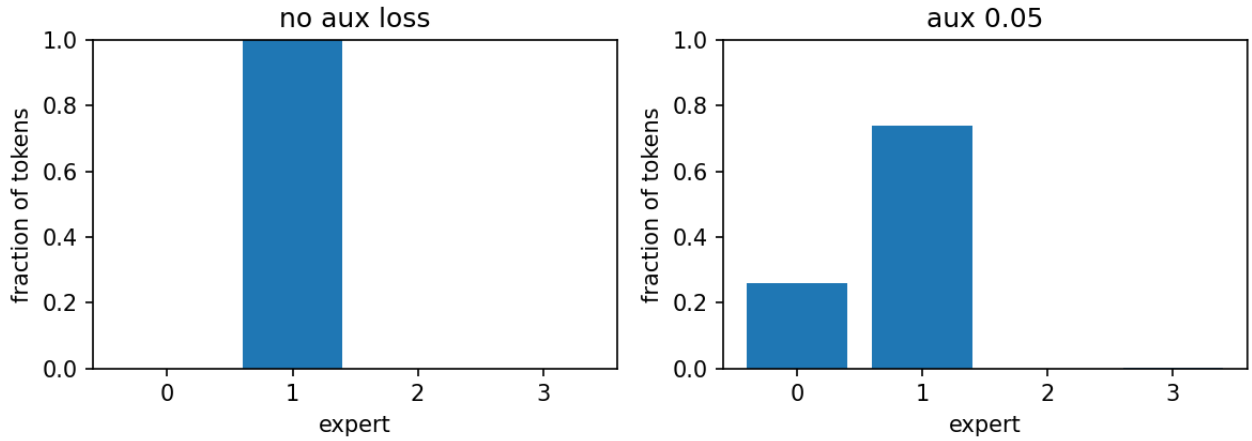
Listing 8 — Mixture-of-experts: top-1 routing, collapse, and load balancing

```python
"""Listing 8: mixture-of-experts -- top-1 routing, expert collapse, and the load-
balancing fix."""
import numpy as np
from sklearn.datasets import load_digits
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(8)

X, y = load_digits(return_X_y=True); X = X/16.0
idx = rng.permutation(len(X)); tr, te = idx[:1400], idx[1400:]
D, H, E, C = 64, 8, 4, 10                                  # 4 experts, top-1 routing

def softmax(z):
    z = z - z.max(-1, keepdims=True); e = np.exp(z); return e/e.sum(-1, keepdims=True)

def train(aux_weight, iters=6000, lr=0.05):
    r = np.random.default_rng(0)
    Wr = r.normal(0, .01, (D, E))                          # router
    W1 = r.normal(0, (2/D)**.5, (E, D, H)); b1 = np.zeros((E, H))
    W2 = r.normal(0, (2/H)**.5, (E, H, C)); b2 = np.zeros((E, C))
    counts = np.zeros(E)
    for it in range(iters):
        i = tr[r.integers(len(tr), size=32)]
        x, yb = X[i], y[i]
        gate = softmax(x@Wr)                               # (B, E)
        sel = gate.argmax(1)                               # top-1: each token to
ONE expert
        counts += np.bincount(sel, minlength=E)
        logits = np.empty((32, C)); h_all = np.empty((32, H))
        for e in range(E):
            m = sel == e
            if m.any():
                h = np.maximum(0, x[m]@W1[e]+b1[e]); h_all[m] = h
                logits[m] = (h@W2[e]+b2[e])*gate[m, e][:, None]   # scale by gate prob
(keeps router in the graph)
        P = softmax(logits)
        d = P.copy(); d[np.arange(32), yb] -= 1; d /= 32
        dgate_sel = np.zeros(32)
        for e in range(E):
            m = sel == e
            if not m.any(): continue
            h = h_all[m]
```

```
            dout = d[m]*gate[m, e][:, None]
            W2[e] -= lr*(h.T@dout); b2[e] -= lr*dout.sum(0)
            dh = (dout@W2[e].T)*(h > 0)
            W1[e] -= lr*(x[m].T@dh); b1[e] -= lr*dh.sum(0)
            dgate_sel[m] = (d[m]*(np.maximum(0, x[m]@W1[e]+b1[e])@W2[e]+b2[e])).sum(1)
        # router grad: through the gate scale (+ optional load-balancing auxiliary
loss)
        dgate = np.zeros((32, E)); dgate[np.arange(32), sel] = dgate_sel
        if aux_weight:
            frac = np.bincount(sel, minlength=E)/32
# Switch-Transformer aux: E * sum(frac_e * mean_gate_e)
            dgate += aux_weight*E*frac[None, :]/32
        dR = dgate*gate - gate*(dgate*gate).sum(1, keepdims=True)   # softmax backward
        Wr -= lr*(x.T@dR)
    # eval
    gate = softmax(X[te]@Wr); sel = gate.argmax(1)
    logits = np.empty((len(te), C))
    for e in range(E):
        m = sel == e
        if m.any():
            logits[m] = (np.maximum(0, X[te][m]@W1[e]+b1[e])@W2[e]+b2[e])*gate[m, e]
[:, None]
    acc = (logits.argmax(1) == y[te]).mean()
    usage = counts/counts.sum()
    total = Wr.size + W1.size + b1.size + W2.size + b2.size
    active = Wr.size + W1[0].size + b1[0].size + W2[0].size + b2[0].size
    return acc, usage, total, active

for aw, tag in [(0.0, "no load-balancing loss"), (0.05, "aux loss weight 0.05  ")]:
    acc, usage, total, active = train(aw)
    print(f"{tag}: acc {acc:.3f}   expert usage {np.round(usage, 2)}")
print(f"parameter accounting: total {total:,}   active per token {active:,}
({100*active/total:.0f}%)")
print("without the aux loss, the rich-get-richer loop starves experts; with it,
capacity actually gets used")

fig, axes = plt.subplots(1, 2, figsize=(8, 3))
for ax, aw, t in [(axes[0], 0.0, 'no aux loss'), (axes[1], 0.05, "aux 0.05")]:
    _, usage, _, _ = train(aw)
    ax.bar(range(E), usage); ax.set(title=t, xlabel='expert', ylabel='fraction of
tokens', ylim=(0, 1))
fig.tight_layout(); fig.savefig("/sessions/intelligent-modest-faraday/mnt/ADVANCE RAG/
AI_ML_Interview_Book/manuscript/figures/ch21/fig_l8_moe.png", dpi=150)
print("figure saved: fig_l8_moe.png")
```

Output:

```
no load-balancing loss: acc 0.952   expert usage [0. 1. 0. 0.]
aux loss weight 0.05  : acc 0.970   expert usage [0.26 0.74 0.   0.  ]
parameter accounting: total 2,696   active per token 866 (32%)
without the aux loss, the rich-get-richer loop starves experts; with it, capacity
actually gets used
figure saved: fig_l8_moe.png
```



Interview questions and answers

Q1. State the Chinchilla result precisely and what it changed.

For a fixed compute budget C ~ 6ND, loss is minimized by scaling parameters N and tokens D roughly EQUALLY — optimal D/N ~ 20 tokens per parameter — versus Kaplan's earlier conclusion favoring much larger N on less data (an artifact of under-tuned small-model baselines and fixed LR schedules). Consequence: Chinchilla (70B, 1.4T tokens) beat Gopher (280B, 300B tokens) at the same compute. What it changed: the field stopped growing parameters and started growing data. The modern caveat: compute-optimal is not INFERENCE-optimal — you pay training once and serving forever, so Llama-class models deliberately overtrain smaller models far past 20 tok/param.

Q2. What does Listing 1's miniature scaling grid show, and where do power laws break?

Excess test loss falls as clean power laws: ~N^-0.13 at fixed data, ~D^-0.37 at fixed parameters (log-log linear over the grid) — the data exponent dominating in the data-starved regime, which is Chinchilla's core point in miniature. Break conditions: (1) overfitting — the 300-sequence run scores 7.80, WORSE than the 6.91 uniform baseline; scaling laws describe the underfit regime and multi-epoch training exits it; (2) approaching irreducible entropy, where the additive constant dominates; (3) data repetition (>~4 epochs adds little); (4) distribution shift between train and eval. Practical use at scale: fit the law on small runs, extrapolate to pick N and D BEFORE the big run.

Q3. Should SFT mask the prompt from the loss? Give the measured two-sided answer.

Listing 2, same data and steps: loss-on-answer-only gives the best task accuracy (0.812 vs 0.734 unmasked) — gradient concentrates on the behavior being taught. But general perplexity: masked 1,645 vs unmasked 169 vs base 133 — the "wasted" prompt loss was replay regularization, and removing it let the task gradient bulldoze the language model. Production answer: mask prompts (you don't want the model learning to generate user turns) AND mix 1-10% pretraining data into SFT batches to supply the replay explicitly. The general principle: what you exclude from the loss, you stop maintaining.

Q4. Write the Bradley-Terry reward-model loss and explain each design choice.

L = -log sigmoid(r_theta(chosen) - r_theta(rejected)). Pairwise rather than absolute scores because humans rank far more reliably than they rate (scale drift, annotator offsets cancel in the difference); sigmoid-of-difference is the Bradley-Terry probability model, making r a latent strength; the head sits on a pretrained encoder (Listing 3: mean-pooled MLM features + scalar head) because preference signal is too scarce to learn language from scratch. Only DIFFERENCES are identified — r has arbitrary offset (production adds a mean-centering or margin term). Result at miniature scale: 95% pairwise accuracy in 300 steps.

Q5. What is reward hacking? Describe the experiment that demonstrates it and the mitigations.

Optimizing a PROXY until the proxy diverges from the target — Goodhart with gradients. Listing 3: RM at ~95% pairwise accuracy on-distribution; greedy coordinate ascent over input tokens finds gibberish scoring +7.69 vs +7.32 for the best real document — and the gibberish is largely anti-topic (hockey) vocabulary, showing how unconstrained the RM is off-manifold. Any optimizer aimed at the RM exits the training distribution immediately (Listing 4's beta=0 run found the same exploit via RL). Mitigations: KL penalty to a reference policy (the leash), RM refresh with on-policy samples between rounds, RM ensembles / uncertainty penalties, length and repetition penalties for the known degenerate directions, and human spot-audits of high-reward outputs.

Q6. Explain the RLHF objective and Listing 4's beta sweep.

max_pi E[r(x)] - beta*KL(pi || pi_ref): maximize learned reward while staying near the reference (SFT) distribution — because r is only trustworthy where pi_ref has support (Q5). Measured dial: beta=0 -> reward +5.11, KL 17, output = team-name salad (the RM exploit, found by RL); beta=0.15 -> +4.51 at KL 7; beta=0.6 -> +3.26 at KL 1.6 with fluent text. No single beta is 'correct' — it prices distribution drift, tuned per run with KL monitoring/early stop. PPO's additions (clipped ratios, value baseline, per-token KL) are variance/stability machinery on the same objective; the miniature uses REINFORCE + moving baseline, the same gradient family.

Q7. Derive DPO from the RLHF objective in three steps.

(1) The KL-constrained optimum has closed form pi*(x) proportional to pi_ref(x)·exp(r(x)/beta). (2) Invert: r(x) = beta·log(pi*(x)/pi_ref(x)) + const — reward is REPARAMETERIZED by the policy itself, the 'your language model is secretly a reward model' step. (3) Substitute into Bradley-Terry: L = -log sigmoid(beta·[log(pi/pi_ref)(chosen) - log(pi/pi_ref)(rejected)]) — a supervised loss over preference pairs; the RM and the RL loop cancel out of the algebra. Gradient weight (1-sigmoid(z)) is largest on wrongly-ranked pairs (Listing 5's coef line). Measured: margin 0 -> +6.38, pair accuracy 0.865, dispreferred ppl 74 -> 174.

Q8. DPO vs PPO-RLHF: when does each win?

DPO: one supervised loss, two model copies (policy + frozen ref), offline data, stable, cheap — the default when you have good preference pairs. Limits: no exploration (it reweights behaviors PRESENT in the data — it cannot discover new ones), sensitive to pair quality/staleness (off-policy pairs describe a different model), and the margin artifact (chosen likelihood can fall too — Listing 5's preferred ppl 70->87; IPO/cDPO patch this). PPO-RLHF: on-policy sampling explores, per-token credit assignment, RM can be refreshed against the current policy (anti-hacking); costs 4 models (policy, ref, RM, value), RL instability, and heavy infra. Practice: DPO-family for most alignment; PPO/GRPO-style when exploration matters — notably reasoning RL with verifiable rewards, where the reward is not a learned proxy.

Q9. What is RLAIF / constitutional AI, and what does it trade?

Replace human preference labels with AI-generated ones. Constitutional AI (Anthropic): (1) SFT phase — model critiques and revises its own outputs against written principles; (2) RL phase — an AI judge picks the better of two responses per the constitution, producing preference data for a reward model (RLAIF). Trades: label cost collapses and scales arbitrarily; consistency beats noisy human raters; principles are explicit and auditable. Costs: inherits the judge model's biases and blind spots systematically (humans' errors are at least uncorrelated), risks self-amplification loops, and judge quality caps the signal. Practice mixes both: human labels where stakes/subtlety are high, AI labels for volume.

Q10. Explain the sign-bug war story from Listing 5 and its general lesson.

A flipped sign in the DPO gradient trained the OPPOSITE preference: margin -35.8, pair accuracy 0.20, and — the diagnostic tell — DISPREFERRED perplexity improving while preferred exploded (78,702). First fix attempt flipped coef AND its uses — net zero change, identical numbers, the classic double-negation trap. Lessons: (1) preference losses fail silently — loss goes DOWN while training the wrong direction, so always plot the margin and per-side likelihoods, not just the loss; (2) after any sign fix, verify numbers actually CHANGED; (3) the cheap invariant: at init (pi == ref) margin must be exactly 0 — Listing 5 prints it. Same discipline as gradient checking, applied to objective direction.

Q11. Write the LoRA parameterization and give the measured case for it.

W = W_0 + (alpha/r)·A·B with A (d x r), B (r x d), r << d; W_0 frozen, only A, B trained; B init zero so training starts exactly at the base model. Chain rule (Listing 6): dL/dA = dL/dW·B^T, dL/dB = A^T·dL/dW — harvest the full-matrix gradient, project. Measured: rank-4 on Wq/Wv matches full fine-tuning (0.766 vs 0.764) training 1.0% of parameters, AND forgets 2.7x less (ppl 217 vs 584) because the backbone cannot move. Deployment: merge A·B into W_0 -> zero inference overhead; or serve many adapters on one backbone. Optimizer-state arithmetic is the real seller: full FT of a 7B model needs ~112 GB of Adam state; rank-16 LoRA needs ~0.5% of that.

Q12. What exactly does QLoRA add over LoRA, and why does it work?

Quantize the FROZEN backbone to 4-bit NF4 (a codebook with quantiles of a normal distribution — matching the empirical weight distribution, better than uniform INT4), keep LoRA adapters in bf16, backprop THROUGH dequantization into the adapters, plus double quantization (quantize the scales themselves) and paged optimizer states. Why it works: the backbone is frozen — quantization error is a fixed perturbation the trainable adapters can compensate, not noise in the gradient path of trained weights; and Listing 7 shows 4-bit per-channel weight error is small to begin with. Result: 65B fine-tuning on a single 48GB GPU at quality matching 16-bit LoRA. The idea generalizes: freeze + compress the big thing, train a small correction in high precision.

Q13. Reproduce Listing 7's quantization ladder and explain each transition.

fp32 ppl 132.9 -> INT8: 133.0/132.8 (per-tensor/per-channel) — free, because 8 bits (~2 decimal digits) exceed the SNR of trained weights. INT4: 171.2 per-tensor vs 144.8 per-channel — 15 levels per sign can't span a whole matrix's dynamic range with one scale; per-channel granularity recovers most of it. INT3: 205/375 — resolution below weight noise floor, error compounds through layers. The mechanism isolated: inject one 20x outlier channel and per-tensor mean error jumps 8.5x (0.016 -> 0.135) — one channel sets the global scale and crushes everyone's resolution. Interview mapping: granularity (per-channel/group-wise) + outlier handling = the entire difference between naive RTN and GPTQ/AWQ/llm.int8.

Q14. GPTQ vs AWQ vs LLM.int8 vs GGUF in one pass.

GPTQ: post-training, column-by-column quantization with second-order error compensation — each column's rounding error is spread over not-yet-quantized columns via the (approximate layer-wise) Hessian; strong INT4/INT3 GPU inference. AWQ: activation-aware — identify the ~1% of weight channels multiplying the largest activations and protect them via per-channel scaling before quantizing; no reconstruction pass, robust across tasks. LLM.int8: INT8 matmuls with OUTLIER activation channels split out and computed in fp16 (the emergent-outlier phenomenon at >6B scale); zero-degradation INT8 serving. GGUF: llama.cpp's file/kernels ecosystem — grouped k-quants (Q4_K etc.) with per-group scales+mins for CPU/edge. All are weight-focused PTQ; choice = hardware + bits target.

Q15. Why does MoE routing collapse without an auxiliary loss? Walk through the feedback loop and the fix.

Rich-get-richer: whichever expert is marginally better at init wins more tokens -> receives more gradient -> improves -> wins more. Listing 8: top-1 routing over 4 experts converges to usage [0, 1, 0, 0]; accuracy pinned at what ONE small expert can do (0.952). Fix: Switch-Transformer auxiliary loss E·sum_e(f_e·p_e) — fraction of tokens routed times mean router probability per expert — minimized by uniform routing; differentiable through p even though top-1 selection isn't. With it: usage spreads, accuracy 0.970. Honest residual: two experts stayed dead — real systems add router noise, capacity factors with token dropping, z-loss on router logits, and still monitor utilization; expert balance is maintained, never solved.

Q16. Explain MoE's parameter/FLOP decoupling with Mixtral's numbers, and its hidden costs.

Per token, only top-k experts run: Mixtral 8x7B holds ~47B parameters but computes ~13B per token (2 of 8 experts) — dense-13B latency with far more capacity (Listing 8's miniature: 32% active). Hidden costs: (1) memory — ALL experts resident, VRAM scales with total parameters (47B), not active; (2) communication — experts sharded across devices need all-to-all token exchange every MoE layer, a bandwidth tax that erodes the FLOP win; (3) load imbalance — hot experts overflow capacity, tokens get dropped or rerouted; (4) fine-tuning instability vs dense; (5) batch-dependence — efficiency needs enough tokens per expert. When MoE wins: pretraining-compute-bound, serving with high throughput demands, batch >> experts. When dense wins: memory-bound serving, small batch, edge.

Q17. Lay out the full modern post-training pipeline in order, with the purpose of each stage.

(1) Pretraining: next-token on trillions of tokens — capability, no behavior. (2) SFT / instruction tuning: thousands-to-millions of demonstrations — format, instruction following, tool syntax (mask prompts, mix pretraining data — Listing 2). (3) Preference optimization: DPO-family on pairs, and/or RLHF with an RM (Listings 3-5) — helpfulness/harmlessness trade-offs, tone, refusals. (4) Reasoning RL (current frontier): RL with VERIFIABLE rewards (unit tests, math checkers) — no proxy RM, so Q5's hacking is structurally reduced. (5) Continuous: RM refresh, red-teaming, targeted patches (often just more SFT data). Orthogonal efficiency choices at each stage: LoRA for 2-3 (Listing 6), quantized serving after (Listing 7). Knowing WHICH stage a behavior comes from is the interview differentiator: capability=1, format=2, judgment=3-4.

Q18. Your DPO-tuned model's win-rate improved but users report it's blander and refuses more. What happened?

Classic preference-optimization side effects. (1) The margin objective punishes anything resembling rejected samples — if rejected pairs skew long/opinionated/risky, the model retreats to safe blandness (Listing 5: even PREFERRED likelihood fell 70->87; mass migrates to a narrow safe region). (2) Judge bias: if preferences came from an AI judge (Q9), verbosity/hedging biases transfer. (3) KL to the SFT reference too weak (beta low) — drift compounds. Diagnostics: per-category refusal rates, response entropy/length distributions vs the SFT checkpoint, margin decomposition (is chosen going up or rejected going down?). Fixes: rebalance pair sourcing, add SFT replay or a positive-likelihood anchor (cDPO/IPO), raise beta, and separate 'harmful' from 'spicy but fine' in labeling guidelines.

Q19. Estimate the memory to fine-tune a 7B model: full FT vs LoRA vs QLoRA.

Full FT with Adam, mixed precision: bf16 weights 14 GB + fp32 master 28 + fp32 m,v 56 = ~98 GB before activations — multi-GPU territory. LoRA r=16 on attention: adapters ~40M params -> trainable state ~0.7 GB; but the FROZEN bf16 backbone still needs 14 GB + activations ~ fits on one 24-40 GB card with gradient checkpointing. QLoRA: backbone NF4 ~3.5 GB + adapters + paged optimizer -> 7B trains on a consumer 12-16 GB GPU; 65-70B on one 48 GB card. Activation memory (batch x seq x layers) rides on top of all three — gradient checkpointing trades ~30% compute to crush it. This arithmetic, done aloud, is a standing systems-interview question.

Q20. Why does RLHF need the reference model at all — why not just maximize the reward?

Because the reward is a PROXY valid only on-distribution. Measured twice in this chapter: Listing 3 — greedy search finds gibberish out-scoring all real text (+7.69 vs +7.32); Listing 4 — beta=0 RL finds the same exploit organically (reward +5.11, KL 17, word salad). The KL term prices every step away from the reference: exploits far off-distribution become unprofitable, and the policy improves within the region where the RM generalizes. Deeper framing: pi_ref encodes everything pretraining+SFT knows about fluent, sensible text; the KL leash is a Bayesian prior over behaviors, and beta sets how much evidence the (noisy, hackable) reward needs to overcome it.

Q21. When is prompt tuning / prefix tuning the right PEFT choice, and why did LoRA mostly win?

Prompt tuning: learn k virtual token embeddings prepended to the input; prefix tuning: learn virtual KV pairs per layer. Right when: thousands of tasks share one frozen backbone with per-task state measured in KILOBYTES (task routing at extreme multi-tenancy), or the API exposes only embeddings. Why LoRA won: quality — soft prompts underperform at small model scale and on hard tasks (they can only steer, not modify computation); capacity is bottlenecked by sequence-length-worth of vectors; they consume context length; and merged LoRA has zero inference overhead while prefixes tax every attention op. Prompt tuning approaches LoRA quality only at very large backbone scale (Lester et al.'s scaling result) — worth citing as the exception.

Q22. What is the 'alignment tax' and where did this chapter measure it?

Capability lost as a side effect of alignment tuning. Sightings here: Listing 2 — masked SFT (best task behavior) exploded general perplexity 12x; Listing 4 — every unit of RM reward cost KL drift, with beta trading them explicitly (+5.11@17 vs +3.26@1.6); Listing 5 — DPO taxed even preferred-text perplexity (70->87); Listing 6 — full FT forgot 2.7x more than LoRA (ppl 584 vs 217). Production mitigations, mapped to each: replay/pretraining-mix (2), KL leash + early stop (4), likelihood anchors (5), PEFT/freezing (6) — plus post-hoc weight interpolation between base and tuned checkpoints. The tax is measurable, budgetable, and mostly evadable with deliberate regularization — treat 'aligned vs capable' as an engineering trade, not a dilemma.

Q23. Design the training plan: adapt a 8B open model into a customer-support agent for a fintech, 50k historical chats, strict tone rules, one A100.

Data: dedupe, PII-scrub, filter to high-CSAT chats; convert to instruction format with prompt masking; hold out by CUSTOMER and time (Chapter 4/25 leakage discipline). Stage 1 — QLoRA SFT (one A100 fits 8B easily; r=16-32, all-linear) on the filtered chats + 5% general instruction data as replay (Listing 2's lesson); evaluate: tone-rubric judge + task completion + regression suite of general capability (alignment-tax watch, Q22). Stage 2 — DPO on pairs mined from CSAT deltas and policy-violation annotations (margin plots per Q10; beta tuned by dispreferred-ppl drift). Ship: merge adapters, quantize AWQ-INT4, measure ppl/task delta (Listing 7's ladder), red-team refusal and PII behaviors. Ongoing: monthly preference refresh from production ratings; canary evals for drift. No PPO — no exploration need, no infra budget.

Q24. Why can reasoning-RL (RLVR) avoid reward hacking that RLHF can't?

RLHF's reward is a LEARNED PROXY of human judgment — hackable off-distribution by construction (Q5). RLVR replaces it with programmatic verification: unit tests pass/fail, math answers check, constraints validate — the reward IS the target on its support, so there's no proxy gap to exploit. Remaining hacks are specification bugs, not model-of-human bugs: gaming weak test suites, special-casing verifier quirks, reward-shaping leaks (e.g., matching answer format without the reasoning) — real but auditable, since the verifier is code you can read and strengthen. Limits: only works where verification exists (code, math, structured tasks); style/helpfulness still needs preference methods; and sparse verifiable rewards bring back RL's exploration and credit-assignment problems — hence process-reward models and dense shaping, which reintroduce proxies deliberately.

Q25. Your RM's pairwise accuracy is 95% but RLHF makes the model worse in human evals. List causes in priority order.

(1) Reward hacking — the policy found off-distribution exploits (Listing 3's +7.69 gibberish; check: sample high-reward outputs and READ them; monitor KL). (2) RM distribution staleness — pairs came from an older policy; current outputs are off-RM-distribution even when good (fix: refresh RM on-policy). (3) Proxy misspecification — raters preferred length/format, RM learned the confound; human evals see through it (check: reward-vs-length correlation). (4) Beta too low — drift outran the leash (Listing 4's dial). (5) Eval mismatch — the human eval measures dimensions (factuality, creativity) the preference data never covered. (6) Value/variance pathologies in PPO itself. The meta-lesson: 95% on-distribution accuracy says NOTHING about behavior under optimization pressure — Listing 3 is the proof.

Q26. Explain why B is initialized to zero in LoRA and what breaks otherwise.

With B=0, A·B=0 at init: the adapted model IS the base model at step 0 — fine-tuning starts from exactly the pretrained function, gradients flow (dL/dB = A^T·dL/dW is nonzero since A is random), and early training makes small, well-conditioned corrections. If both A and B were random: A·B adds an O(1) random perturbation to W_0 BEFORE any training — you begin from a damaged model (instant perplexity spike), and the first gradients must undo noise rather than learn task structure; with both zero, dA and dB are both zero — a saddle, nothing trains. Same design instinct as ch14's residual-branch downscaling and ch16's pre-norm: initialize new machinery to identity, learn deviations.

Q27. This chapter's alignment-pipeline results in one sweep.

Scaling: excess loss ~ N^-0.13, D^-0.37; 300-seq run overfits past uniform (Listing 1). SFT: masking 0.812 vs 0.734, but ppl 1,645 vs 169 — prompt loss was replay (Listing 2). RM: 95% pairwise; greedy search finds gibberish at +7.69 > best real +7.32 (Listing 3). RLHF beta dial: +5.11/KL17 word salad -> +3.26/KL1.6 fluent (Listing 4). DPO: margin +6.38, pairs 0.865, dispreferred ppl 74->174, preferred taxed 70->87; sign bug trained the reverse and only the margin plot caught it (Listing 5). LoRA: 0.766 vs 0.764 at 1.0% params, forgetting 217 vs 584 (Listing 6). Quantization: INT8 free, INT4 145/171 per-channel/tensor, INT3 dies; 20x outlier -> 8.5x per-tensor error (Listing 7). MoE: collapse [0,1,0,0] without aux loss; 0.952 -> 0.970 with it; 32% params active (Listing 8).

Q28. A skeptical senior engineer says alignment is 'just fine-tuning with extra steps.' Give the technically-grounded rebuttal.

The steps exist because supervised fine-tuning alone cannot express the objective. SFT teaches 'imitate these outputs' — but the target is 'outputs humans PREFER', a ranking over the model's own candidate space, including candidates no demonstrator wrote; preference losses (Listings 3-5) optimize that directly. Second, naive optimization of any learned objective self-destructs — Listing 3's RM is 95% accurate and still maximized by gibberish; the KL-constrained formulation (Listing 4) is what makes optimizing a proxy survivable, and DPO is its closed-form — none of that machinery appears in vanilla fine-tuning. Third, the failure modes are distinct and measurable: reward hacking, margin-induced blandness, alignment tax — each with its own diagnostics and regularizers (Q18, Q22). 'Fine-tuning with extra steps' is true in the sense that GPS is 'clocks with extra steps' — the extra steps are the solution to the actual problem.